Mentimeter

Laborator 13

Exceptii

Instructions

Mentimeter

Mentimeter

Exceptie

eveniment care se produce in timpul executiei si perturba flowul normal

Mentimeter

Exemple de exceptii

- citim un fisier care nu exista
- nu avem permisiuni sa citim un fisier
- acces concurent la un fisier

1

Mentimeter

```
try {
    System.out.println(Integer.parseInt( s: "of"));
} catch (NumberFormatException e) {
    System.out.println("of eroare: " + e.getMessage());
}
```

Exemplu try catch

Mentimeter

Principiu try catch

- se executa secvential instructiunile din blocul try
- la aparatia unei exceptii => se abandoneaza restul instructionilor
- + se sare direct la blocul de catch corespunzator

1

Mentimeter

Exceptii in Java

Cand apare o eroare => creeaza un obiect exceptie si il paseaza catre runtime system (throwing)

Mentimeter

Obiect exceptie

- tipul exceptiei
- stiva de apeluri (stack trace) - unde a intervenit exceptia, reprezentat sub forma unui lant de metode

Mentimeter

```
Integer integer = null;
if (integer == null) {
    throw new Exception("Alo nu null");
}
```

Throwing exceptions

Mentimeter

Observatii

- exceptiile sunt tot obiecte (se pot crea folosind new)
- aruncarea unei exceptii se face folosind throw
- clasa Exception = parintele majoritatii claselor exceptie

Mentimeter

Exemple de exceptii standard

- IndexOutOfBondsException
- NullPointerException
- NoSuchElementException

Mentimeter

```java
public static Integer makeFancyInteger() throws Exception {
  Integer integer = null;
  if (integer == null) {
    throw new Exception("Alo nu null");
  }
  return integer;
}

public static void main(String[] args) {
  try {
    System.out.println(Main.makeFancyInteger());
  } catch (Exception e) {
    e.printStackTrace();
  }
}
```

Catching exceptions



Mentimeter

Observatii

- daca o functie genereaza o exceptie trebuie sa o prinda sau sa o adauge in semnatura
- daca nu prindem exceptia in main => se inchide programul

Mentimeter

```java
public static int makeFancyDouble() {
  int value = (int) (Math.random() % 2);
  if (value == 0) {
    throw new IndexOutOfBoundsException("ceva");
  }
  if (value == 1) {
    throw new NullPointerException("altceva");
  }
  return value;
}

public static void main(String[] args) {
  try {
    System.out.println(Main.makeFancyDouble());
  } catch (IndexOutOfBoundsException exception) {
    exception.printStackTrace();
  } catch (NullPointerException exception) {
    exception.printStackTrace();
  }
}
```

Catch/throw la mai multe tipuri de exceptii

Mentimeter

```java
public static int makeFancyDouble() {
  int value = (int) (Math.random() % 2);
  if (value == 0) {
    throw new IndexOutOfBoundsException("ceva");
  }
  if (value == 1) {
    throw new NullPointerException("altceva");
  }
  return value;
}

public static void main(String[] args) {
  try {
    System.out.println(Main.makeFancyDouble());
  } catch (IndexOutOfBoundsException | NullPointerException exception) {
    exception.printStackTrace();
  }
}
```

Catch/throw la mai multe tipuri de exceptii

Mentimeter

Observatii

- pot arunca mai multe tipuri de exceptii
- pot prinde diferit exceptiile in functie de tipul lor

Mentimeter

```
try {
  System.out.println(Main.makeFancyDouble());
  try {
    System.out.println(Main.makeFancyDouble());
  } catch (IndexOutOfBoundsException | NullPointerException exception) {
    exception.printStackTrace();
  }
} catch (IndexOutOfBoundsException | NullPointerException exception) {
  exception.printStackTrace();
}
```

Blocuri try imbricate

Mentimeter

```java
try {
    System.out.println(Main.makeFancyDouble());
} catch (IndexOutOfBoundsException | NullPointerException exception) {
    exception.printStackTrace();
} finally {
    System.out.println("Always here");
}
```

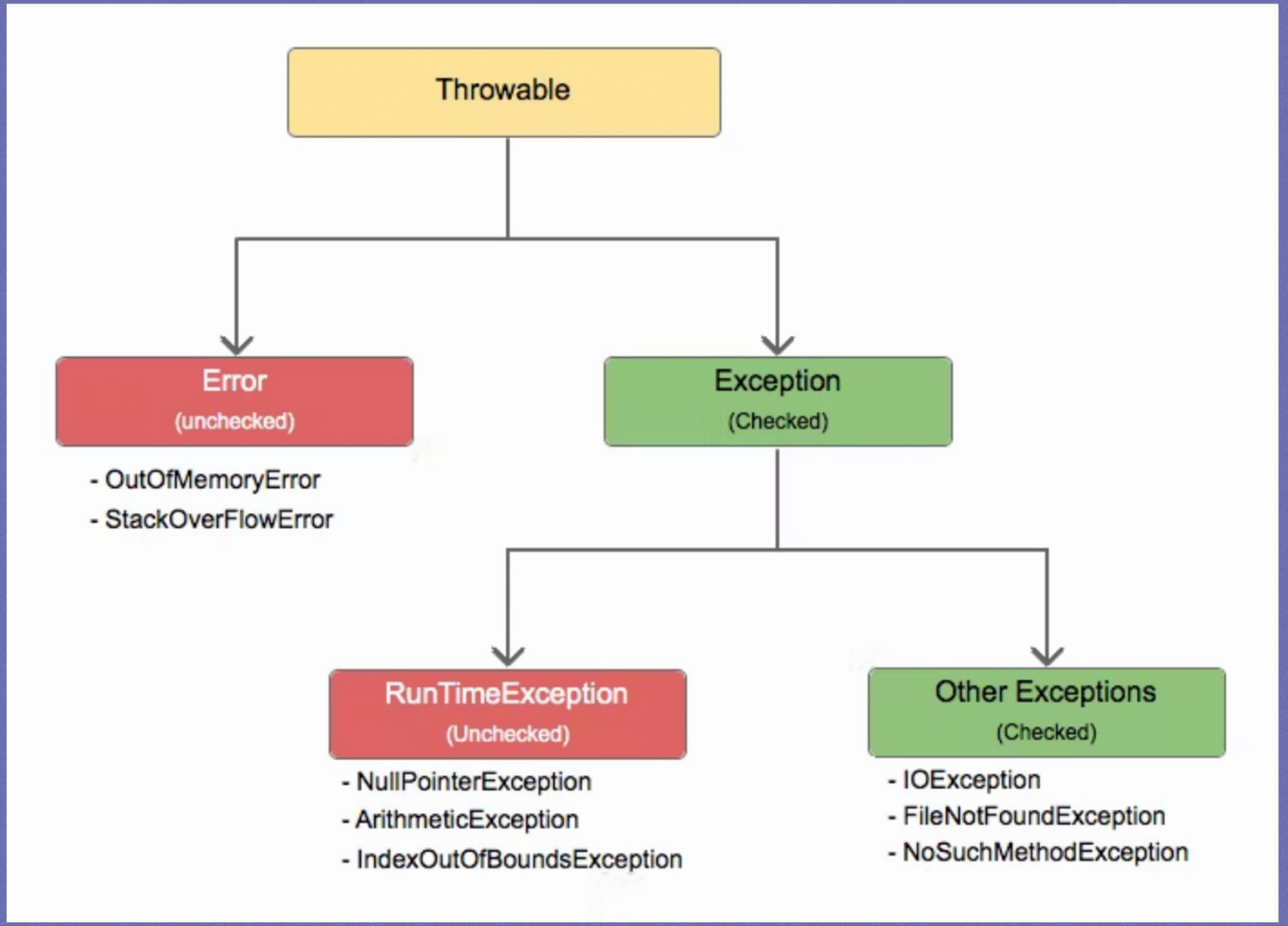
Blocul finally

Mentimeter

Blocul finally

- executat mereu indiferent daca se returneaza sau nu vreo exceptie

Mentimeter



Ierarhie exceptii

Mentimeter

Throwable

- superclasa tuturor erorilor si exceptiilor din Java
- numai obiectele care extind clasa asta pot fi aruncate cu throw si prinse cu catch

Mentimeter

Clasa Exception -> checked exceptions

- prinse cu blocuri try catch
- factori interni: trebuie sa ne asiguram ca tratam fiecare caz cum trebuie

Mentimeter

Error

- erori declansate de factori externi
- conditii exceptionale pe care nu le pot anticipa
- (in general) nu pot continua programul daca apar (ex. OutOfMemoryError)

Mentimeter

Runtime Exceptions

- conditii exceptionale pe care nu le pot anticipa
- daar declansate de factori interni
- pot fi prinse cu try catch, dar mai bine trebuie rezolvat bugul care produce valorile dubioase

Mentimeter

Checked vs unchecked

- Checked: Exception fara RuntimException
- Unchecked: Error + RuntimeException + subclasele lor
- cele unchecked nu e recomandat sa fie prinse cu try catch

Mentimeter

Compilare checked vs unchecked exceptions

Compilatorul tipa numai daca avem exceptii checked care nu sunt declarate in semnatura cu throws si nici nu sunt prinse in blocuri try catch

Mentimeter

```
class FancyException extends Exception {}

class DoubleFancyException extends FancyException {}
```

Exceptii custom

Mentimeter

```
public static int makeFancyDouble() throws FancyException, DoubleFancyException {
  int value = (int) (Math.random() % 2);
  if (value == 0) {
    throw new FancyException();
  }
  if (value == 1) {
    throw new DoubleFancyException();
  }
  return value;
}
```

Throw

Mentimeter

```java
public static void main(String[] args) {
    try {
        System.out.println(Main.makeFancyDouble());
    } catch (FancyException e) {
        e.printStackTrace();
    } catch (DoubleFancyException e) {
        e.printStackTrace();
    }
}
```

Catch

Mentimeter

Atentie super mare la ordine

- se face match pe primul catch ok
- DoubleFancyException extinde FancyException
- daca pun FancyException in primul catch => nu o sa prind niciodata DoubleFancyException

Mentimeter

```
try {
    System.out.println(Main.makeFancyDouble());
} catch (DoubleFancyException e) {
    e.printStackTrace();
} catch (FancyException e) {
    e.printStackTrace();
}
```

De la cel mai specific la cel mai generic

Mentimeter

Exceptiile în contextul mostenirii

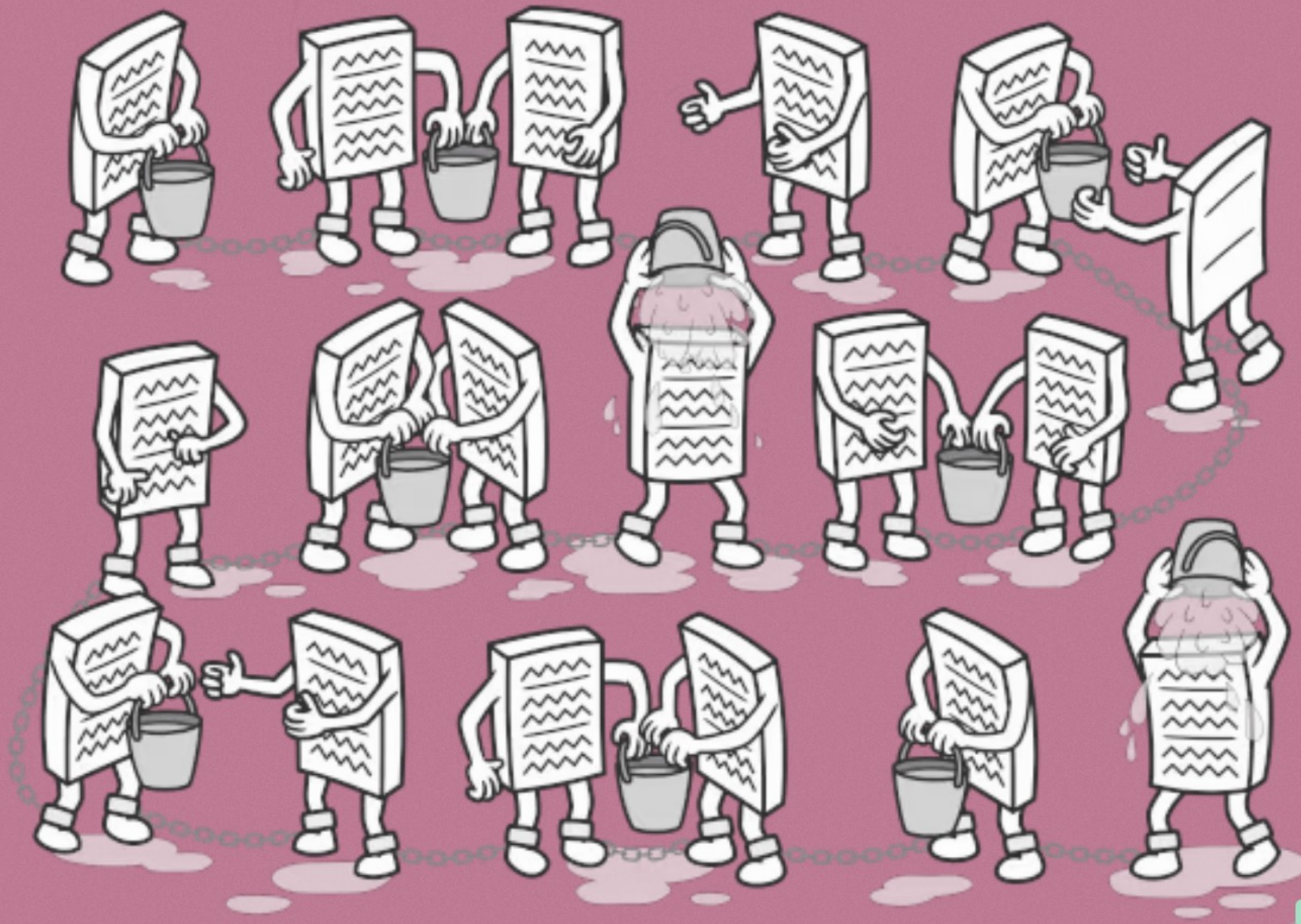
Metodele suprascrise (overriden) pot arunca numai exceptiile specificate de metoda din clasa de baza sau subclase

Mentimeter

Chain-of-responsibility Pattern

- un lant de receivers
- primesc pe rand exceptiile si decid daca sa le trateze sau nu
- le trimit mai departe urmatorului din lista

Mentimeter



1